

Here's a **clean, interview-ready table** that covers **everything required for API development**, along with **tools, technologies, and their purpose**. This works well for **real-world projects, system design discussions, and 10–15+ years experience interviews**.

□ API Development – Requirements, Tools & Technologies

Category	What is Required	Purpose / Why Needed	Common Tools & Technologies
API Type & Style	REST / GraphQL / gRPC / SOAP	Defines how clients communicate with the server	REST, GraphQL, gRPC, SOAP
Programming Language	Backend language	Implements business logic	Java, Python, Node.js, Go, C#
Framework	Web framework	Simplifies request handling & routing	Spring Boot, Django, Flask, Express, FastAPI
HTTP Protocol	HTTP/HTTPS	Communication protocol	HTTP/1.1, HTTP/2, HTTPS
Request Methods	GET, POST, PUT, DELETE, PATCH	Defines CRUD operations	REST standards
Data Format	JSON / XML / Protobuf	Data exchange format	JSON (most common), XML, Protobuf
API Design Standards	Naming & conventions	Consistency and readability	RESTful conventions
Versioning	API versions	Backward compatibility	/v1, /v2, headers
Input Validation	Request validation	Prevents invalid data	Hibernate Validator, Joi, Pydantic
Error Handling	Standard error responses	Improves debugging & UX	HTTP status codes, custom error models
Authentication	Identity verification	Secures APIs	JWT, OAuth2, Basic Auth
Authorization	Access control	Role-based permissions	RBAC, ABAC
Security	API protection	Prevents attacks	HTTPS, CORS, CSRF, Rate limiting

API Gateway	Central entry point	Routing, auth, throttling	Kong, NGINX, AWS API Gateway
Database	Data persistence	Stores application data	MySQL, PostgreSQL, MongoDB
ORM / Data Access	DB interaction	Reduces boilerplate code	Hibernate, JPA, Prisma, SQLAlchemy
Caching	Performance improvement	Reduces DB load	Redis, Memcached
Pagination & Filtering	Large data handling	Improves response efficiency	Page, Limit, Cursor-based pagination
Logging	Request & error tracking	Debugging & monitoring	Logback, Log4j, Winston
Monitoring	System health	Detects failures early	Prometheus, Grafana
Tracing	Distributed tracing	Debug microservices	OpenTelemetry, Jaeger
Documentation	API reference	Developer usability	Swagger / OpenAPI
Testing (Unit)	Code correctness	Validates logic	JUnit, Mockito, PyTest
Testing (Integration)	End-to-end validation	Ensures system works together	RestAssured, Postman
Mocking	API simulation	Independent testing	WireMock, MockServer
CI/CD	Automated pipeline	Faster deployments	Jenkins, GitHub Actions
Version Control	Source code management	Collaboration & history	Git, GitHub, GitLab
Build Tools	Dependency management	Builds & packages app	Maven, Gradle, npm
Containerization	Environment consistency	Easy deployment	Docker
Orchestration	Container management	Scalability & resilience	Kubernetes
Cloud Platform	Hosting & scaling	Infrastructure management	AWS, Azure, GCP

Load Balancing	Traffic distribution	High availability	NGINX, AWS ALB
Rate Limiting	Traffic control	Prevents abuse	API Gateway, Redis
API Throttling	Usage control	Protects backend	Token bucket, leaky bucket
Secrets Management	Secure credentials	Prevents leaks	Vault, AWS Secrets Manager
Environment Management	Dev / QA / Prod separation	Stable deployments	Profiles, ENV variables
Compliance	Data & security laws	Legal adherence	GDPR, HIPAA (domain-based)

❑ **Typical API Tech Stack Example (Real Project)**

Layer	Example Stack
API Style	REST
Backend	Java + Spring Boot
Security	JWT + OAuth2
Database	PostgreSQL
Cache	Redis
Docs	Swagger
Testing	JUnit + RestAssured
CI/CD	Jenkins
Deployment	Docker + Kubernetes
Cloud	AWS

❑ **Interview Tip (❑ Frequently Asked)**

“What are the most important things in API development?”

Answer:

- ✓ API design
 - ✓ Security
 - ✓ Performance
 - ✓ Scalability
 - ✓ Documentation
 - ✓ Observability (logs, metrics, tracing)
-

If you want, I can also provide:

- ✓ **API Development Architecture Diagram**
- ✓ **REST vs GraphQL vs gRPC comparison**
- ✓ **Production-ready API checklist**
- ✓ **API interview Q&A (senior / architect level)**